

KhataAI

Master

Document

Final Pre-
Implementation

Checklist

The definitive
blueprint
governing
the
development,
architecture,
and launch
criteria for
the KhataAI
platform.

Table of Contents

- | | |
|---------------------------|--------------------------------|
| 1. Project Status | 9. Provider Abstractions |
| 2. Final Technology Stack | 10. Coding Standards |
| 3. Final Architecture | 11. Project Board |
| 4. Final Database Status | 12. MVP vs Future Features |
| 5. Final Modules | 13. Ready-to-Develop Checklist |
| 6. Final API Modules | 14. Final Recommendations |
| 7. Final Folder Structure | 15. Conclusion |
| 8. Development Roadmap | |

1. Project Status

Status: DESIGN FROZEN. READY FOR IMPLEMENTATION.

All architectural planning, database schematics, API routing contracts, deployment topologies, and testing strategies have been formally defined and locked.

2. Final Technology Stack

LAYER	TECHNOLOGY
Framework & Language	FastAPI, Python 3.12+
Database (Primary)	MongoDB Community (NoSQL)
Caching & Queues	Redis
Background Workers	Celery
Artificial Intelligence	Gemini 1.5 Pro / GPT-4o, OpenAI Whisper (STT)
Storage & Infrastructure	MinIO (S3-compatible), Docker Compose, NGINX

3. Final Architecture

KhataAI utilizes a **Feature-Based Architecture** integrated with an **Event-Driven Background Flow**. The system follows a strict Voice-First execution loop: *Voice → Whisper STT → LLM Intent Parser → AI Orchestrator → Business Service → MongoDB → Event Publisher → Celery Worker*.

4. Final Database Status

The database schema is built on a **Progressive ERP** design utilizing L1 (Essential), L2 (Business), and L3 (Enterprise) fields.

- **Identity:** users, stores
- **Ledgers:** parties (Customers/Suppliers), products, inventory_logs
- **Financials:** transactions (Immutable), expenses
- **Intelligence:** ocr_documents, voice_history, audit_logs

5. Final Modules

- | | |
|------------------------------------|--------------------------------|
| ✓ Authentication & Access | ✓ Expense Tracking |
| ✓ Store Profile & Settings | ✓ Voice AI Processing |
| ✓ Customer & Supplier Directory | ✓ OCR Document Extraction |
| ✓ Product & Inventory Catalog | ✓ Business Analytics & Reports |
| ✓ Transaction & Invoice Processing | ✓ Notifications & Alerts |

6. Final API Modules

All endpoints enforce the `/api/v1/` prefix, strict JSON payloads, JWT validation, and the standard envelope: `{success, message, data, errors}`.

- `/auth/*` (OTP, Verification, Refresh)
- `/store/*` (Profile, Settings, Logos)
- `/customers/*` & `/suppliers/*` (Ledgers, CRUD)
- `/products/*` & `/inventory/*` (Stock IO, Catalog)
- `/transactions/*` (Invoices, Bills, Reversals, Payments)
- `/voice/*` & `/ocr/*` (STT streaming, Document parsing)
- `/ai/*` (Chat, Insights)

7. Final Folder Structure

```
app/  
├─ core/           # App configs, security, interfaces  
├─ database/       # MongoDB client & Redis pools  
├─ middleware/     # JWT auth, rate limiting  
├─ common/         # Shared utilities  
├─ workers/        # Celery tasks (invoice, notifications)  
├─ ai/             # Orchestrator, chat, speech, ocr  
├─ auth/           # Domain feature module  
├─ store/          # Domain feature module  
├─ customer/       # Domain feature module  
├─ product/        # Domain feature module  
├─ transaction/    # Domain feature module  
└─ ...
```

8. Development Roadmap (Implementation Order)

- | | |
|--------------------------|----------------------------|
| 1. Authentication (OTP) | 7. Inventory |
| 2. Store Settings | 8. Purchases |
| 3. Users & Roles | 9. Invoices & Transactions |
| 4. Customers | 10. Payments |
| 5. Categories & Products | 11. Expenses |
| 6. Suppliers | 12. Files & Storage |

- 13. Notifications
- 14. Analytics
- 15. Voice Processing

- 16. OCR Extraction
- 17. AI Chat
- 18. Admin Controls

9. Provider Abstractions

To avoid vendor lock-in, the system relies on frozen Python ABC interfaces in `app/core/interfaces/`. Concrete implementations depend on:

- `LLMProvider` (Extract intents, chat contexts)
- `StorageProvider` (Upload files, sign URLs)
- `OTPPProvider` (Dispatch verification codes)
- `NotificationProvider` (WhatsApp, Email dispatches)
- `OCRProvider` (Process invoice bytes)

10. Coding Standards

✔ Strict Pydantic payload validation.

✔ Dependency Injection via FastAPI `Depends()`.

✔ Centralized exception handling preventing stack-trace leaks.

✔ Asynchronous I/O across all DB/Network layers.

✔ Type hinting enforced via `mypy`.

✔ Repository Pattern isolating DB logic from Services.

11. Project Board (Epics)

Development is split into 19 Epics mapping directly to the Implementation Roadmap. Each Epic represents an independent feature branch, thoroughly tested via unit and API suites before merging to ``main``.

12. MVP vs Future Features

MVP SCOPE (PHASE 1)	FUTURE FEATURES (PHASE 3+)
OTP Authentication & Basic RBAC	Advanced Multi-Store & Employee Roles
Product, Customer & Supplier Ledgers	Predictive Inventory Replenishment
Standard Voice Transcription (STT)	Autonomous AI Agent Workflows
Invoice Generation & PDF Export	Vector-based RAG Search
Single-node Docker Compose Deployment	AWS ECS/EKS Kubernetes Scaling

13. Ready-to-Develop Checklist

- | | |
|---------------------------------------|--|
| ✓ Modules defined | ✓ Database collections & fields locked |
| ✓ MVP scope defined | ✓ API routing contracts approved |
| ✓ Future roadmap defined | ✓ Folder structure created |
| ✓ Feature-based architecture approved | ✓ Testing strategy finalized |

14. Final Recommendations

Stop Planning. Start Coding. Adhere strictly to the established abstractions to maintain flexibility. Treat the transaction ledger as completely immutable. Do not bypass the Service layer to write directly to MongoDB from the AI Orchestrator.

15. Conclusion

The architectural foundation for KhataAI is resilient, scalable, and tailored explicitly for the voice-first SME market in India. The design phase is closed. Tomorrow begins with initializing the FastAPI repository, connecting MongoDB, and deploying the OTP Authentication Epic.